



UNIVERSIDADE VEIGA DE ALMEIDA
CIÊNCIA DA COMPUTAÇÃO E ENGENHARIA DA COMPUTAÇÃO
CAMPUS TIJUCA

Laboratório de Desenvolvimento de Software

Bruno Ferreira da Costa - 1240114845	Bruno Lourenço Queiroz da Silva - 1240120417
Carlos Augusto da Silva Souza - 12401020417	Eduardo Rodrigues Gomes - 1240208119
Gabriel Ribeiro - 1240113883	Gabriel Vasconcelos - 121012929
Guilherme Ribeiro Alves - 1240200753	Jamilly Dias Deodato - 1240205458
João Vitor Hermes Fonseca Coelho - 1240112457	João Victor Pereira dos Reis - 1240111812
Lucas Rodrigues Correia - 120105219	Matheus de Souza - 1240105219
Maurício Gonçalves Simões Júnior - 1230103599	Rafael Matias Alonso Carvalhal - 1240107511
Victor Gusmão Moreira Vieira - 1260114525	

2026

Bruno Ferreira da Costa - 1240114845	Bruno Lourenço Queiroz da Silva - 1240120417
Carlos Augusto da Silva Souza - 12401020417	Eduardo Rodrigues Gomes - 1240208119
Gabriel Ribeiro - 1240113883	Gabriel Vasconcelos - 121012929
Guilherme Ribeiro Alves - 1240200753	Jamilly Dias Deodato - 1240205458
João Vitor Hermes Fonseca Coelho - 1240112457	João Victor Pereira dos Reis - 1240111812
Lucas Rodrigues Correia - 120105219	Matheus de Souza - 1240105219
Maurício Gonçalves Simões Júnior - 1230103599	Rafael Matias Alonso Carvalhal - 1240107511
Victor Gusmão Moreira Vieira - 1260114525	

Trabalho apresentado à disciplina de programação em Laboratório de Desenvolvimento de Software dos cursos de Ciência da computação e Engenharia da computação da Universidade Veiga de Almeida.

Orientador: Prof. Paulo Andrade

Rio de Janeiro

Sumário

1. Resumo	7
2. Introdução	7
3. Fundamentação Teórica	8
3.1 Arquitetura Cliente-Servidor e Desacoplamento de Camadas	8
3.2 Desenvolvimento de APIs RESTful e Engenharia de Back-end	9
3.3 Modelagem Relacional e Persistência de Dados (ORM)	9
3.4 Segurança em Sistemas Web e Integração com Serviços Externos	9
4. Desenvolvimento do Sistema	10
4.1 Arquitetura da Aplicação.	10
4.2 Banco de Dados	11
4.3 Backend	12
4.4 Frontend	14
4.5 API REST	15
4.6 Docker e Docker Compose	17
5. Documentação das Telas	17
5.1 Visão Geral do Sistema	18
5.2 Mapa de Telas e Fluxo de Navegação	18
5.3 Documentação das Telas	19
5.3.1 Tela de Login	19
Descrição	19
Componentes e Elementos	19
Regras	19
Captura de Tela	20
5.3.2 Tela de Cadastro	20
Descrição	21
Componentes e Elementos	21

Regras	21
Captura de Tela	22
5.3.3 Dashboard — Estado Vazio	23
Descrição	23
Componentes e Elementos	23
Observações	23
Captura de Tela	23
5.3.4 Dashboard — Com Filmes Cadastrados	25
Descrição	25
Componentes e Elementos	25
Captura de Tela	26
5.3.5 Busca de Filme via TMDb	27
Descrição	27
Componentes e Elementos	27
Regras	27
Captura de Tela	28
5.3.6 Formulário de Adição de Filme	29
Descrição	29
Componentes e Elementos	29
Regras	29
Capturas de Tela	30
5.3.7 Formulário de Edição de Filme	31
Descrição	32
Componentes e Elementos	32
Regras	32
Capturas de Tela	33
5.3.8 Exclusão de Filme	34
Descrição	34

Componentes e Elementos	34
Regras	34
Capturas de Tela	35
5.4 Referência dos Endpoints da API	36
5.4.1 Autenticação	36
5.4.2 Filmes (privado)	36
5.4.3 Busca TMDb (privado)	36
5.5 Estrutura do Banco de Dados	37
Tabela: usuarios	37
Tabela: filmes	37
5.6 Resumo das Telas	37
6. Organização Ágil e Backlog	38
6.1 Divisão de Responsabilidades	38
6.2 Backlog de Funcionalidades	39
7. Versionamento e GitHub	40
7.1 Padrão de Commits	40
7.2 Pull Requests	41
7.3 Divisão de Trabalho na Equipe	41
8. Resultados	42
8.1 Funcionalidades Entregues	42
8.2 Indicadores de Qualidade	42
9. Conclusão	43
10. Referências	44

1. Resumo

Este trabalho apresenta o projeto e a implementação de uma aplicação web full-stack sob a arquitetura cliente-servidor, projetada para o gerenciamento, organização e análise estatística de catálogos cinematográficos pessoais. A interface de usuário (front-end) foi desenvolvida utilizando a biblioteca React integrada ao ecossistema Vite, assegurando uma experiência responsiva, dinâmica e de alta fidelidade visual. O ecossistema de back-end baseia-se em uma API REST assíncrona implementada em Python com o framework FastAPI, responsável pelo processamento de requisições HTTP, execução das regras de negócio e validação rigorosa de dados via schemas. A persistência de dados utiliza o motor SQLite gerenciado pelo ORM SQLAlchemy, estabelecendo um relacionamento de um para muitos (1:N) que isola estritamente o catálogo de cada usuário autenticado. A segurança da aplicação é garantida por meio de criptografia de credenciais e autenticação baseada em Tokens JWT para a proteção de rotas privadas e prevenção de acessos não autorizados. Adicionalmente, o software consome a API externa TMDb (The Movie Database) para a automação na busca de metadados de mídias, implementa um sistema estruturado de tratamento de erros e renderiza um painel estatístico (dashboard) analítico com indicadores agregados sobre o acervo do usuário. O projeto consolida os conceitos fundamentais de modularização, tratamento de exceções, segurança digital, interoperabilidade de sistemas e engenharia de software aplicada.

Palavras-chave: API REST, FastAPI, React, SQLAlchemy, Docker, CRUD, Python, JavaScript.

2. Introdução

O desenvolvimento de aplicações web é hoje uma das competências centrais da Engenharia de Computação. Projetos práticos que integram backend, banco de dados e frontend permitem ao estudante compreender o ciclo completo de uma aplicação real, desde a modelação dos dados até a entrega de uma interface funcional ao usuário final.

Diante desse contexto, este projeto propõe o desenvolvimento de uma API RESTful para gerenciamento de filmes. A escolha do domínio é deliberada: é familiar, compreensível e permite focar a atenção na arquitetura e nas tecnologias, sem a complexidade de um domínio de negócio sofisticado.

A equipe, composta por 15 integrantes, adotou uma divisão de trabalho baseada em camadas da aplicação, utilizando Git e GitHub como ferramentas de versionamento e colaboração. O processo de desenvolvimento identificou e resolveu problemas reais, como o comportamento do Docker Compose ao mapear bancos SQLite, devidamente documentados neste relatório.

Este documento está organizado nas seções exigidas: após esta introdução, são apresentados a fundamentação teórica, o desenvolvimento detalhado do sistema, a documentação das telas, a organização ágil, o versionamento, os resultados obtidos e as conclusões.

3. Fundamentação Teórica

3.1 Arquitetura Cliente-Servidor e Desacoplamento de Camadas

O modelo arquitetural cliente-servidor distribui o processamento da aplicação em camadas independentes, otimizando o consumo de recursos e facilitando a manutenção do código. No desenvolvimento web moderno, o desacoplamento entre a interface gráfica (front-end) e a lógica de negócios (back-end) permite o desenvolvimento paralelo e agiliza a escalabilidade do ecossistema.

- **Front-end Componentizado (React + Vite):** A biblioteca React baseia-se na criação de interfaces orientadas a componentes modulares e na manipulação de estado por meio de uma árvore DOM virtual, o que reduz o custo computacional de re-renderização na tela. O uso do Vite como ferramenta de build otimiza o empacotamento de arquivos através de módulos ESM (ECMAScript Modules) nativos, entregando alto desempenho e carregamento instantâneo durante o desenvolvimento.

3.2 Desenvolvimento de APIs RESTful e Engenharia de Back-end

O padrão de arquitetura REST (Representational State Transfer) define um conjunto de restrições para a criação de serviços web baseados nos métodos nativos do protocolo HTTP (GET, POST, PUT, DELETE). A comunicação padronizada por meio de payloads em formato JSON garante a interoperabilidade e a independência de plataformas.

- **Framework FastAPI:** Operando de forma nativa com concorrência assíncrona em Python, o FastAPI destaca-se pelo alto desempenho computacional. Ao utilizar a tipagem estática da linguagem, o framework automatiza a validação de esquemas de dados em tempo de execução e mitiga inconsistências no recebimento de parâmetros na API.

3.3 Modelagem Relacional e Persistência de Dados (ORM)

A persistência de dados em arquivos relacionais assegura a integridade, a consistência e a durabilidade das informações transacionadas pela aplicação ao longo do tempo.

- **Mapeamento Objeto-Relacional (SQLAlchemy):** O uso de uma camada ORM (Object-Relational Mapping) elimina a necessidade de escrita direta de instruções SQL

complexas dentro do código-fonte. O ORM atua traduzindo os registros das tabelas do banco de dados em classes e objetos manipuláveis na linguagem de programação. Essa abstração aumenta a segurança do banco contra ataques de injeção de código (SQL Injection).

- **Relacionamento Um-para-Muitos (1:N):** A estrutura de banco de dados utiliza chaves estrangeiras para conectar a tabela de usuários à tabela de filmes. Esse vínculo lógico garante que um usuário específico possa possuir múltiplos itens persistidos em sua coleção pessoal, enquanto impede restritamente que contas de terceiros acessem ou alterem dados sem autorização prévia.

3.4 Segurança em Sistemas Web e Integração com Serviços Externos

A segurança em arquiteturas distribuídas exige que o servidor seja capaz de validar a identidade e o escopo de permissões do cliente a cada nova requisição, sem a necessidade de manter estados ativos de sessão na memória (stateless).

- **Autenticação por Tokens JWT (JSON Web Token):** O padrão JWT permite a transmissão de informações de identidade de forma compacta e assinada digitalmente. Após o processo de login com credenciais previamente criptografadas, o servidor emite um token assinado. O cliente armazena esse artefato e o envia nos cabeçalhos HTTP subsequentes, permitindo a validação instantânea do acesso a rotas protegidas pelo sistema.

- **Consumo de APIs de Terceiros:** A integração com serviços externos (como a API do TMDB) demonstra o princípio da reutilização de dados na engenharia de software. Ao delegar a consulta e o fornecimento de grandes acervos de dados de mídia a um provedor especializado, a aplicação local reduz sua carga de armazenamento e processamento, mantendo o foco do código estritamente na experiência do usuário e em suas estatísticas de consumo.

4. Desenvolvimento do Sistema

4.1 Arquitetura da Aplicação

O sistema foi desenvolvido utilizando uma arquitetura cliente-servidor, composta por um frontend responsável pela interface gráfica e um backend responsável pelo processamento das requisições e gerenciamento dos dados.

O frontend foi desenvolvido utilizando React e Vite, proporcionando uma interface moderna e dinâmica para os usuários. Já o backend foi implementado utilizando FastAPI, framework Python voltado para o desenvolvimento de APIs REST.

A persistência dos dados é realizada por meio do banco de dados SQLite, utilizando o SQLAlchemy como ORM para mapear as entidades da aplicação para tabelas relacionais.

Além disso, o sistema realiza integração com a API TMDb (The Movie Database), permitindo a pesquisa e obtenção de informações sobre filmes.

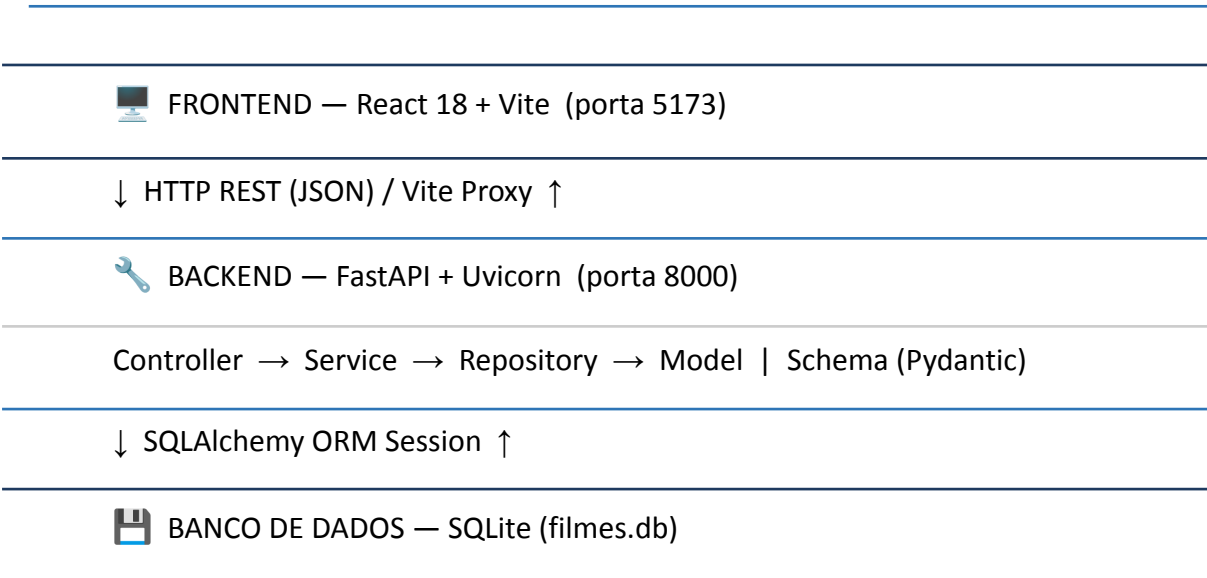


Figura 1 — Diagrama de arquitetura geral da aplicação

4.2 Banco de Dados

O sistema utiliza o banco de dados SQLite para armazenamento persistente das informações dos usuários e dos filmes cadastrados. A comunicação entre a aplicação e o banco é realizada por meio do SQLAlchemy, responsável pelo mapeamento objeto-relacional (ORM).

O banco de dados foi estruturado para armazenar as informações necessárias para autenticação dos usuários e gerenciamento do catálogo de filmes.

Tabela Usuários:

A tabela usuários é responsável pelo armazenamento das informações de autenticação dos usuários cadastrados no sistema.

Campo	Tipo	Descrição
Id	Integer	Identificador único do usuário
username	String	Nome de usuário utilizado para login
hashed_password	String	Senha criptografada armazenada no banco

Tabela Filmes:

A tabela filmes armazena os filmes cadastrados pelos usuários no catálogo pessoal.

Campo	Tipo	Descrição
id	Integer	Identificador único do filme
titulo	String	Nome do filme
genero	String	Gênero do filme
ano	Integer	Ano de lançamento

Cam	T	Descrição
po	ipo	
nota	Float	usuário
post		
er	String	pôster
usua		
rio	String	cadastro
data		
_cadastro	String	Data de cadastro do filme

Relacionamento entre as Tabelas:

Cada usuário pode cadastrar diversos filmes em seu catálogo pessoal. Dessa forma, existe um relacionamento de um para muitos (1:N), onde um usuário pode possuir vários filmes cadastrados.

4.3 Backend

O backend da aplicação foi desenvolvido utilizando o framework FastAPI, responsável pelo processamento das requisições HTTP, autenticação dos usuários, gerenciamento dos filmes cadastrados e integração com serviços externos.

A estrutura do backend foi organizada em módulos, seguindo uma separação de responsabilidades que facilita a manutenção e evolução do sistema.

Controllers:

Os controllers são responsáveis por receber as requisições do frontend, processar os dados recebidos e encaminhar as operações para as camadas responsáveis pela lógica de negócio.

A aplicação possui três controllers principais:

- Auth Controller: responsável pelo cadastro e autenticação dos usuários.
- Filme Controller: responsável pelas operações de cadastro, consulta, atualização e remoção de filmes.
- TMDB Controller: responsável pela integração com a API TMDB para pesquisa de filmes.

Autenticação:

O sistema utiliza autenticação baseada em tokens JWT (JSON Web Token). Após realizar o login com sucesso, o usuário recebe um token de acesso que deve ser enviado nas requisições protegidas.

Esse mecanismo garante maior segurança, permitindo que apenas usuários autenticados acessem funcionalidades restritas da aplicação.

Regras de Negócio:

Foram implementadas diversas regras para garantir a integridade e segurança dos dados:

- Apenas usuários autenticados podem acessar o catálogo de filmes.
- Cada usuário pode visualizar apenas os filmes cadastrados em sua própria conta.
- Não é permitido alterar ou excluir filmes pertencentes a outros usuários.
- O sistema verifica a existência do filme antes de realizar operações de atualização ou exclusão.
- A pesquisa de filmes na API TMDB exige um termo de busca válido.

Validações:

As validações dos dados são realizadas por meio dos schemas da aplicação, garantindo que as informações recebidas estejam no formato esperado antes de serem processadas.

Também são realizados tratamentos de erros para situações como:

- Usuário ou senha inválidos.
- Tentativa de acesso não autorizado.
- Filme não encontrado.
- Falhas durante consultas externas.

Integração com Serviços Externos:

A aplicação realiza integração com a API TMDb (The Movie Database), permitindo que os usuários pesquisem filmes e obtenham informações como título, gênero, ano de lançamento e imagem de capa para inclusão no catálogo pessoal.

4.4 Frontend

O frontend da aplicação foi desenvolvido utilizando React em conjunto com o Vite, proporcionando uma interface moderna, responsiva e de fácil utilização. O objetivo principal foi oferecer uma experiência intuitiva para que os usuários possam gerenciar seu catálogo de filmes de forma simples e eficiente.

A comunicação com o backend é realizada por meio de requisições HTTP para a API REST desenvolvida em FastAPI.

Tela de Login:

A tela de login permite que usuários previamente cadastrados realizem a autenticação no sistema. O acesso é realizado por meio de nome de usuário e senha, sendo validado pelo backend antes da liberação do acesso às funcionalidades da aplicação.

Tela de Cadastro:

A tela de cadastro possibilita a criação de novas contas de usuário. Durante o processo, as informações fornecidas são enviadas para a API, que realiza as validações necessárias antes de armazenar os dados no banco de dados.

Catálogo de Filmes:

A tela principal do sistema apresenta o catálogo pessoal de filmes do usuário. Nessa área é possível visualizar os filmes cadastrados, consultar informações relevantes e gerenciar o conteúdo armazenado.

Pesquisa de Filmes:

O sistema permite a pesquisa de filmes utilizando a API TMDb. Ao informar o nome de um filme, são exibidos resultados contendo informações como título, gênero, ano de lançamento e imagem de capa.

Dashboard de Estatísticas:

A aplicação disponibiliza um painel de estatísticas que apresenta informações relacionadas ao catálogo do usuário. Entre os indicadores exibidos estão quantidade de filmes cadastrados, média das avaliações e distribuição dos gêneros presentes no catálogo.

Navegação e Intergração:

A navegação entre as páginas ocorre de forma dinâmica, utilizando os recursos do React. Todas as operações realizadas pelo usuário são enviadas para a API REST, que processa as informações e retorna os dados necessários para atualização da interface.

4.5 API REST

A comunicação entre o frontend e o backend é realizada por meio de uma API REST desenvolvida utilizando o framework FastAPI. A API é responsável por processar as requisições da aplicação, realizar validações, aplicar as regras de negócio e retornar os dados necessários para a interface do usuário.

As informações são transmitidas no formato JSON, seguindo os padrões utilizados no desenvolvimento de aplicações web modernas.

Endpoints de Autenticação:

Os endpoints de autenticação são responsáveis pelo cadastro e acesso dos usuários ao sistema.

Método	Endpoint	Descrição
POST	/register	Realiza o cadastro de um novo usuário
POST	/login	Realiza a autenticação e gera um token JWT

Endpoints de Filmes:

Os endpoints de filmes permitem o gerenciamento completo do catálogo pessoal do usuário.

Método	Endpoint	Descrição
GET	/filmes	Lista todos os filmes do usuário autenticado
GET	/filmes/{id}	Retorna os dados de um filme específico
POST	/filmes	Adiciona um novo filme ao catálogo
PUT	/filmes/{id}	Atualiza os dados de um filme
DELETE	/filmes/{id}	Remove um filme do catálogo

Endpoint de Pesquisa:

A aplicação também disponibiliza integração com a API TMDb para pesquisa de filmes.

Método HTTP	Endpoint	Descrição
GET	/tmdb/search	Realiza a pesquisa de filmes utilizando a API TMDb

Segurança da API:

As rotas protegidas utilizam autenticação baseada em JWT (JSON Web Token). Após realizar o login, o usuário recebe um token de acesso que deve ser enviado nas requisições subsequentes.

Esse mecanismo garante que apenas usuários autenticados possam acessar os recursos protegidos da aplicação, além de impedir o acesso aos dados de outros usuários.

Respostas e Tratamento de Erros:

A API retorna códigos HTTP apropriados para cada situação. Entre os principais códigos utilizados estão:

- 200: Operação realizada com sucesso;
- 401: Usuário não autenticado ou credenciais inválidas;
- 403: Acesso não autorizado ao recurso solicitado;
- 404: Registro não encontrado;
- 500: Erro interno do servidor.

Esses retornos permitem que o frontend trate adequadamente cada situação e forneça feedback claro ao usuário.

4.6 Docker e Docker Compose

- Serviço app: imagem python:3.11-slim, porta 8000, volume ./data:/app/data para persistência do SQLite.
- Serviço frontend: imagem node:18-alpine, porta 5173, com volume para hot-reload.

 Problema resolvido: Docker Compose criava diretório vazio ao mapear arquivo SQLite diretamente (./filmes.db:/app/filmes.db), causando OperationalError no SQLAlchemy. Solução: mapear diretório inteiro (./data:/app/data) e atualizar DATABASE_URL para sqlite:///app/data/filmes.db.

5. Documentação das Telas

Catálogo de Filmes — Full-Stack

React + FastAPI + SQLite

5.1 Visão Geral do Sistema

O Catálogo de Filmes é uma aplicação web full-stack que permite aos usuários gerenciar diversos filmes. O sistema oferece autenticação segura com JWT, integração com a API do TMDB para busca de filmes, operações completas de CRUD e um painel completo com informações dos filmes.

Front-end	React 18 + Vite (http://localhost:5173)
Back-end	Python 3.11 + FastAPI (http://localhost:8000)

Banco de Dados	SQLite (filmes.db)
Autenticação	JWT (Bearer Token)
API Externa	TMDB — The Movie Database
Design	Dark Theme inspirado em plataformas de streaming

5.2 Mapa de Telas e Fluxo de Navegação

O sistema é composto por duas rotas principais: a tela de autenticação (pública) e o catálogo (protegida por login). O fluxo de navegação é linear e controlado por token JWT armazenado no localStorage.

Login / Cadastro → Catálogo (Dashboard)

Rotas de acesso não autenticado: /login e /register. Rota protegida: / (dashboard principal).

5.3 Documentação das Telas

5.3.1 Tela de Login

Identificador	TELA-01
Rota	/login (padrão ao acessar a aplicação)
Acesso	Pública — não precisa de autenticação
Endpoint	POST /auth/login

Descrição

Tela inicial da aplicação. Permite que usuários já cadastrados acessem seu catálogo pessoal inserindo usuário e senha. Após a autenticação, o token JWT é armazenado e o usuário é redirecionado ao Dashboard.

Componentes e Elementos

- Ícone da aplicação (claquete com estrela, cor roxa)
- Título: "Seu Catálogo de Filmes"
- Subtítulo: "Acesse seu Acervo"
- Campo de texto — Usuário (placeholder: "Seu usuário")
- Campo de senha — Senha (placeholder: "Sua senha")
- Botão primário roxo: "ACESSAR CATÁLOGO" — dispara POST

/auth/login

- Botão secundário: "CRIAR UMA CONTA" — redireciona para a tela de Cadastro

Regras

- Ambos os campos são obrigatórios
- Em caso de credenciais inválidas, o back-end retorna 401 Unauthorized
- Ao autenticar com sucesso, o token JWT é salvo no localStorage e o usuário é redirecionado ao Dashboard

Captura de Tela

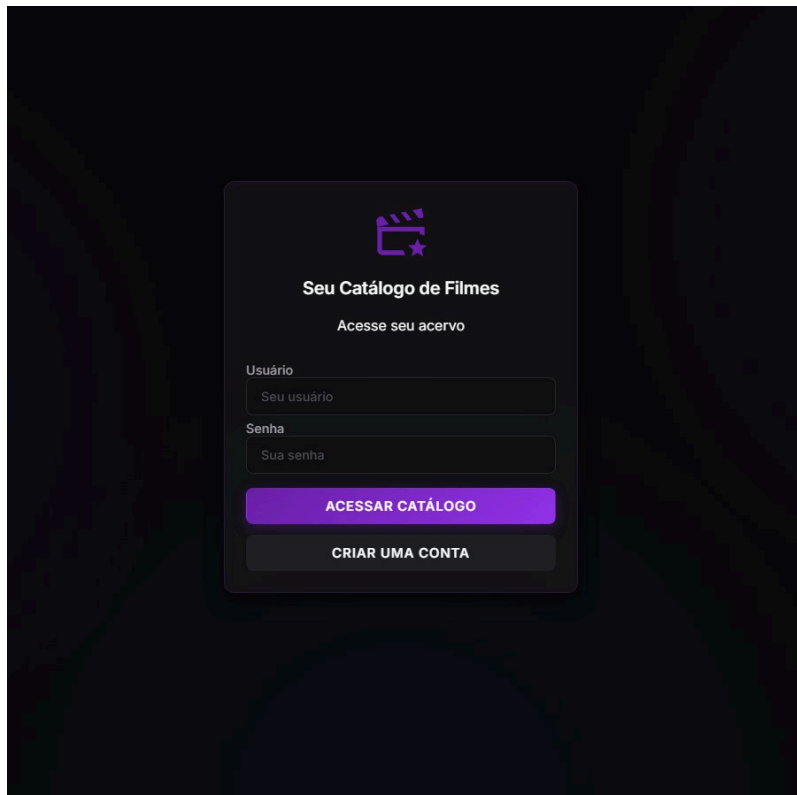


Figura 1 — Tela de Login

5.3.2 Tela de Cadastro

Identificador	TELA-02
Rota	/register
Acesso	Pública — não requer autenticação
Endpoint	POST /auth/register

Descrição

Permite a criação de uma nova conta para o usuário. O cadastro exige o nome do usuário e definição de senha com confirmação. Após o cadastro na plataforma, o usuário é redirecionado para plataforma.

Componentes e Elementos

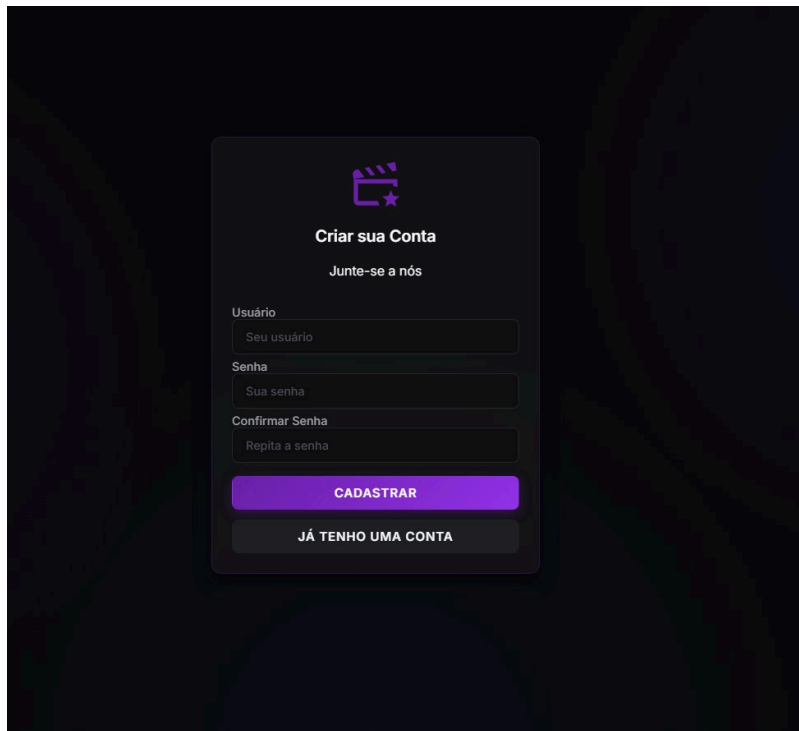
- Ícone da aplicação (claquete com estrela, cor roxa)
- Título: "Criar sua Conta"
- Subtítulo: "Junte-se a Nós"
- Campo de texto — Usuário (placeholder: "Seu usuário")
- Campo de senha — Senha (placeholder: "Sua senha")
- Campo de senha — Confirmar Senha (placeholder: "Repita a senha")
- Botão primário roxo: "Cadastrar" — dispara POST /auth/register
- Botão secundário: "Já tenho uma conta" — redireciona para tela de

Login

Regras

- Todos os campos são obrigatórios
- O nome de usuário deve ser único no banco de dados
- A senha é armazenada de forma criptografada via bcrypt
- Os campos "Senha" e "Confirmar Senha" devem ser idênticos

Captura de Tela



The image shows a registration form centered on a dark, textured background. At the top of the form is a purple icon of a clapperboard with a star. Below the icon, the text 'Criar sua Conta' is displayed in white, followed by the subtitle 'Junte-se a nós'. The form contains three input fields: 'Usuário' with the placeholder 'Seu usuário', 'Senha' with the placeholder 'Sua senha', and 'Confirmar Senha' with the placeholder 'Repita a senha'. At the bottom of the form are two buttons: a prominent purple button labeled 'CADASTRAR' and a smaller, lighter button labeled 'JÁ TENHO UMA CONTA'.



Criar sua Conta

Junte-se a nós

Usuário

Seu usuário

Senha

Sua senha

Confirmar Senha

Repita a senha

CADASTRAR

JÁ TENHO UMA CONTA

Figura 2 — Tela de Cadastro

5.3.3 Dashboard — Estado Vazio

Identificador	TELA-03
Rota	/ (protegida por JWT)
Acesso	Privado — requer autenticação
Endpoint	GET /filmes

Descrição

Estado inicial do Dashboard exibido quando o usuário não possui filmes cadastrados. O painel fica oculto e a seção "Meus Filmes" exibe uma mensagem de catálogo vazio. A seção "Adicionar Filme" permanece disponível para o usuário iniciar seu catálogo.

Componentes e Elementos

- Header com logo "Catálogo", saudação personalizada ("Olá, [usuário]!") e botão "SAIR"
- Painel "Adicionar Filme" com campo de busca TMDB
- Seção "Meus Filmes" com botão "ATUALIZAR"
- Estado vazio: ícone de claquete + mensagem "Seu catálogo está vazio."

Observações

- O painel analítico (estatísticas e gráfico) só aparece quando há ao menos um filme cadastrado
- O botão "SAIR" encerra a sessão limpando o token do localStorage

Captura de Tela

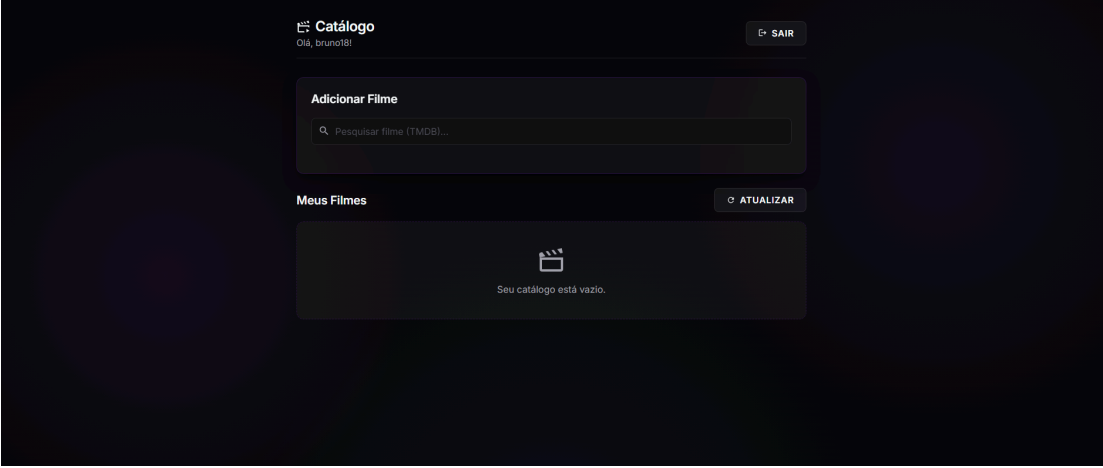


Figura 3 — Dashboard com catálogo vazio

5.3.4 Dashboard — Com Filmes Cadastrados

Identificador	TELA-04
Rota	/ (protegida por JWT)
Acesso	Privado — requer autenticação
Endpoint	GET /filmes

Descrição

Estado principal do Dashboard após o usuário cadastrar filmes. Exibe o painel completo com estatísticas calculadas dinamicamente e a grade de cards com os filmes do usuário, ordenados do mais recente para o mais antigo.

Componentes e Elementos

- Header com logo, saudação e botão "SAIR"
- Painel "Adicionar Filme" com campo de busca TMDb
- Painel analítico — bloco de estatísticas:
 - Total Assistido: contagem total de filmes cadastrados
 - Horas Assistidas (aprox.): estimativa calculada como total × duração do filme arredondando (1h40min = 2h)
 - Média de Notas: média aritmética das notas de todos os filmes (escala 0–10)
- Painel analítico — Gráfico "Seus Gêneros Favoritos": gráfico de rosca (Recharts/PieChart) com os gêneros mais frequentes, cores semânticas por gênero e legenda lateral
- Seção "Meus Filmes" com botão "ATUALIZAR" e grade de cards
- Card de filme: pôster, título, gêneros, ano, data de cadastro, avaliação em estrelas (0–5), ícone de editar (lápis) e ícone de excluir (lixeira vermelha)

Captura de Tela

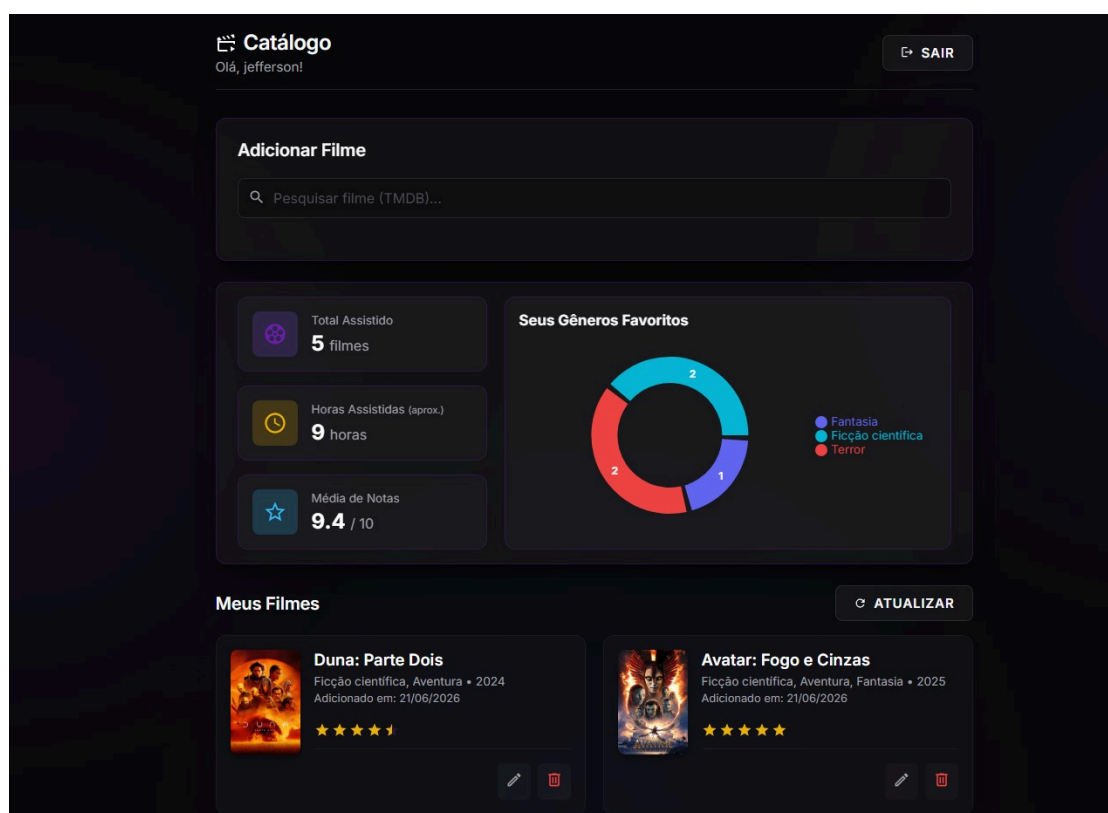


Figura 4 — Dashboard com filmes cadastrados e painel analítico

5.3.5 Busca de Filme via TMDB

Identificador

TELA-05

Localização	Painel "Adicionar Filme" no Dashboard
Endpoint	GET /tmdb/search?query=...
Requer Auth	Sim — Bearer Token JWT no header

Descrição

Campo de busca integrado à API do TMDB. Ao digitar pelo menos 3 caracteres, o sistema aguarda 500ms (debounce) e exibe um dropdown com até 10 resultados. Cada resultado exibe pôster, título, ano e gêneros. Ao selecionar um resultado, o formulário de avaliação é preenchido de forma automática.

Componentes e Elementos

- Campo de texto com ícone de lupa (placeholder: "Pesquisar filme (TMDB)...")
 - Ícone de carregamento durante a busca
 - Dropdown de resultados com cards contendo: miniatura do pôster, título em negrito, ano e gêneros

Regras

- Busca ativada a partir 3 ou mais caracteres
- Debounce de 500ms evita chamadas excessivas à API
- Resultados limitados a 10 itens por busca
- Ao selecionar um filme, o dropdown fecha e o formulário de avaliação é exibido
- A requisição exige token JWT válido no header Authorization

Captura de Tela

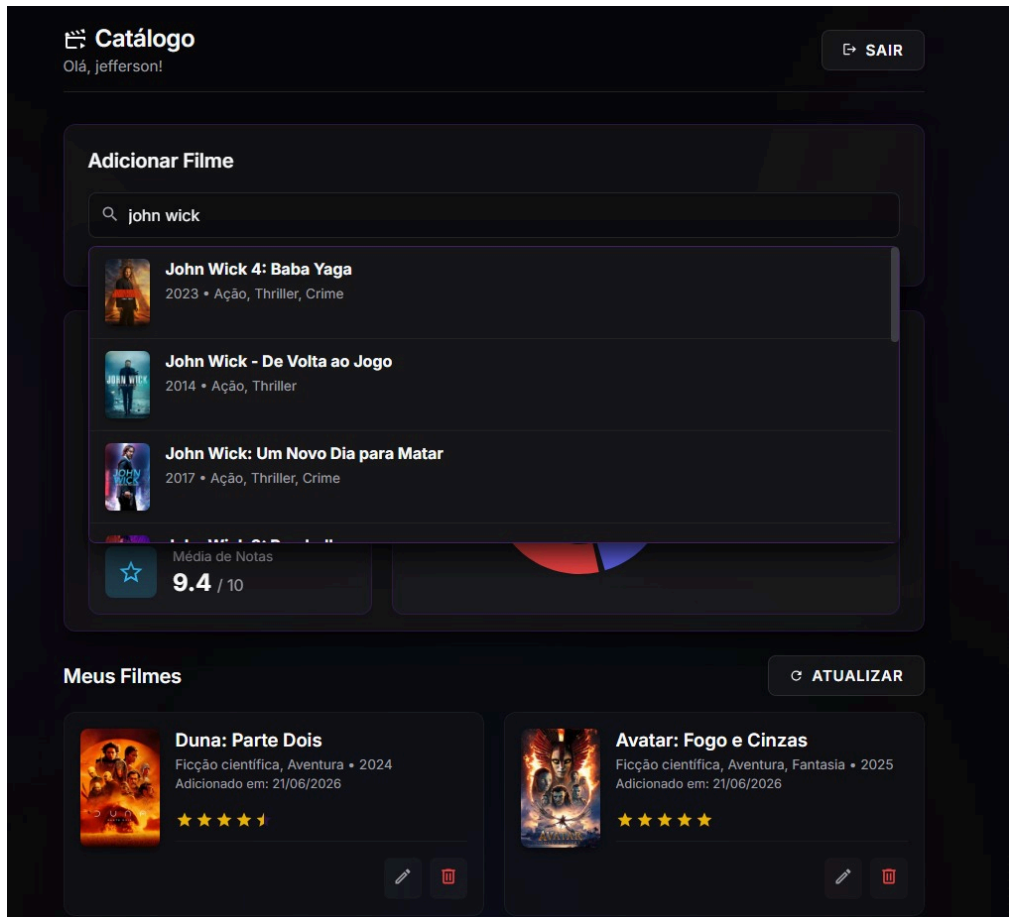


Figura 5 — Dropdown de busca TMDb com resultados

5.3.6 Formulário de Adição de Filme

Identificador	TELA-06
Localização	Painel "Adicionar Filme" — após selecionar resultado TMDb
Endpoint	POST /filmes
Requer Auth	Sim — Bearer Token JWT

Descrição

Formulário exibido após o usuário selecionar um filme no dropdown do TMDb. Exibe as informações do filme (pôster, título, ano, gêneros) e permite que o usuário avalie o filme de 0 a 5 estrelas. Um envio sem selecionar um filme é bloqueado pelo front-end.

Componentes e Elementos

- Título do painel: "Adicionar Filme"
- Campo de busca (funciona mesmo com filme selecionado, permitindo o usuário alterar o filme durante avaliação)
- Card de pré-visualização: pôster (60×90px), título com ano e gênero em roxo
- Seção "Sua Avaliação": componente de estrelas interativo (5 estrelas, suporte a meia estrela)
 - Valor numérico da nota exibido ao lado das estrelas (ex: "5.0")
 - Botão primário roxo: "ADICIONAR" (ícone de check) — dispara POST /filmes
 - Botão secundário: "CANCELAR" — limpa formulário e exibe mensagem "Cancelado."

Regras

- O título é obrigatório — botão "ADICIONAR" é bloqueado se nenhum filme foi selecionado.
- A nota padrão é 0 caso o usuário não interaja com as estrelas.

- O proprietário do filme é vinculado automaticamente via token JWT.
- A data de cadastro é preenchida automaticamente pelo front-end ao enviar.

Capturas de Tela

A captura de tela mostra a interface de um aplicativo com o título "Catálogo" e o nome de usuário "Olá, Usuário!". No topo direito, há um botão "SAIR". O formulário principal, intitulado "Adicionar Filme", contém uma barra de pesquisa com o texto "Pesquisar filme (TMDB)...". Abaixo, há uma sugestão de filme: "John Wick 4: Baba Yaga (2023)" com o gênero "Ação, Thriller, Crime". A seção "Sua Avaliação" apresenta cinco estrelas vazias e o valor "0.0". No final do formulário, há dois botões: "ADICIONAR" (verde) e "CANCELAR" (cinza). Abaixo do formulário, há uma seção "Meus Filmes" com um botão "ATUALIZAR". O conteúdo desta seção é vazio, exibindo apenas um ícone de claqueta e o texto "Seu catálogo está vazio."

Figura 6 — Formulário de adição com filme selecionado e avaliação

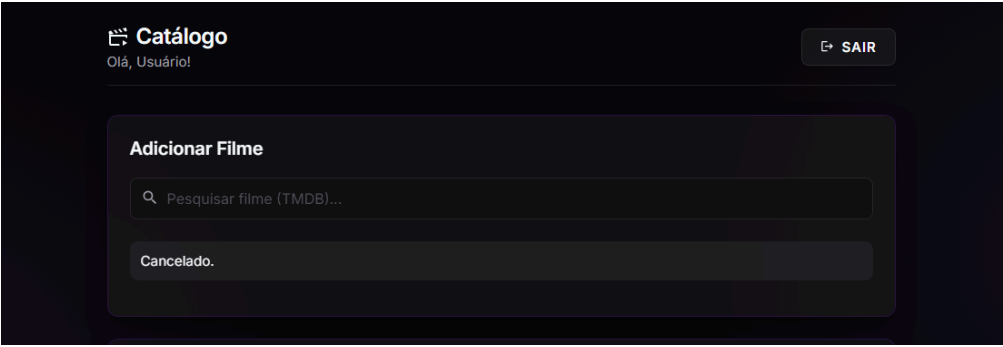


Figura 7 — Feedback "Cancelado." após cancelar a adição

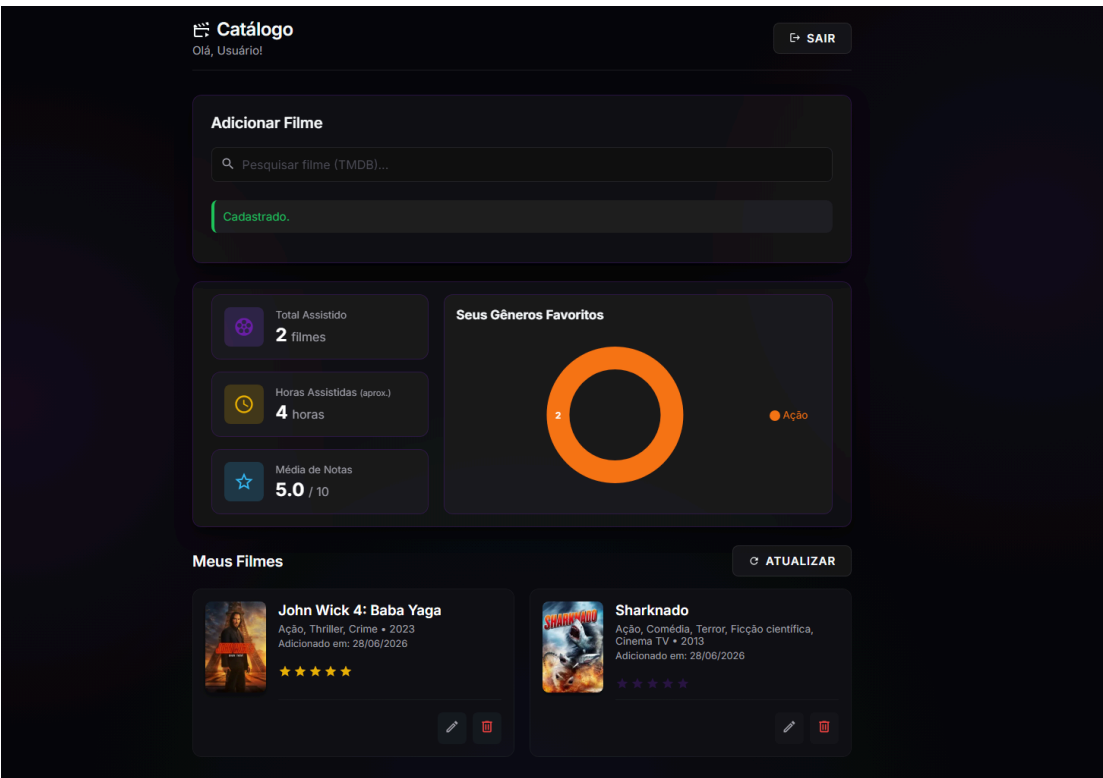


Figura 8 — Feedback "Cadastrado." após adicionar com sucesso

5.3.7 Formulário de Edição de Filme

Identificador	TELA-07
Localização	Painel superior do Dashboard (substitui "Adicionar Filme")

Ativação	Clique no ícone de lápis (editar) em um card de filme
Endpoint	PUT /filmes/{id}
Requer Auth	Sim — Bearer Token JWT

Descrição

Ao clicar no ícone de lápis em um filme, o painel superior alterna para o modo de edição. O título muda para "Editar Filme" e o formulário é preenchido com os dados atuais do filme. O campo de busca TMDb fica oculto neste modo. A página rola automaticamente para o topo para facilitar a edição.

Componentes e Elementos

- Título do painel: "Editar Filme" (diferencia visualmente do modo de adição)
- Card de pré-visualização: pôster, título com ano e gênero em roxo (somente leitura)
- Seção "Sua Avaliação": componente de estrelas com a nota atual pré-carregada
- Botão primário roxo: "SALVAR" (ícone de check) — dispara PUT /filmes/{id}
- Botão secundário: "CANCELAR" — descarta alterações

Regras

- Somente a nota pode ser alterada (título, gênero, ano e pôster vêm do TMDb).
- O envio atualiza apenas o registro do usuário autenticado (validação via JWT no back-end).
- Após o salvamento, exibe um feedback escrito "Atualizado" em verde.

Capturas de Tela

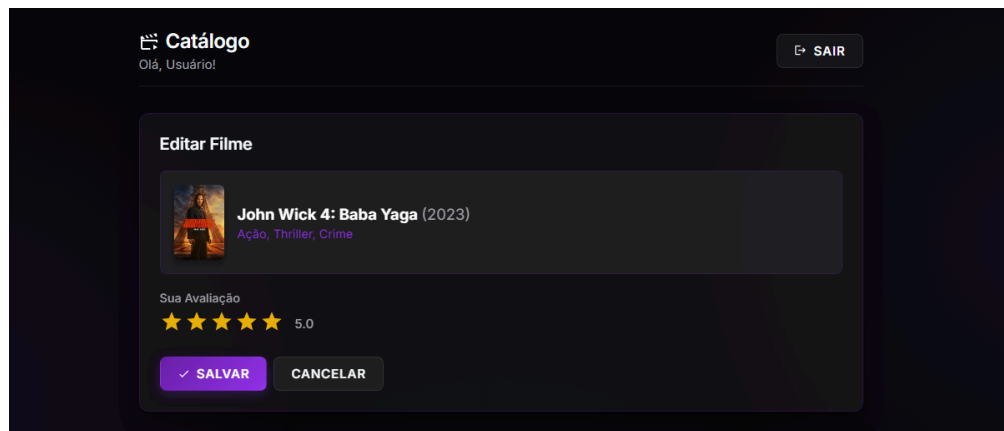


Figura 9 — Formulário de edição com nota pré-carregada

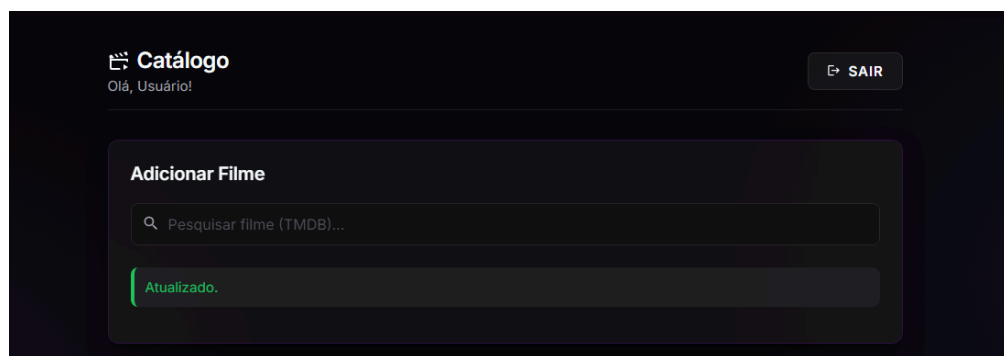


Figura 10 — Feedback "Atualizado." após salvar edição

5.3.8 Exclusão de Filme

Identificador	TELA-08
Ativação	Clique no ícone de lixeira vermelha em um card de filme
Endpoint	DELETE /filmes/{id}
Requer Auth	Sim — Bearer Token JWT

Descrição

Ao clicar no ícone de lixeira em um filme, o navegador exibe uma caixa de diálogo de confirmação com o nome do filme. O usuário deve confirmar a ação clicando em "OK" para que a exclusão seja processada. Após a exclusão, o filme é removido da grade e um feedback é exibido.

Componentes e Elementos

- Dialog nativo do navegador: "Excluir [nome do filme]?" com botões "OK" e "Cancelar".
- Feedback de sucesso: mensagem "Removido." exibida em verde no painel superior.
- Grade de filmes atualizada automaticamente após a exclusão.

Regras

- A exclusão requer confirmação explícita via dialog.
- Se o usuário clicar em "Cancelar" no dialog, nenhuma ação é executada.
- O back-end valida via JWT que o filme pertence ao usuário antes de excluir.
- Se o filme em edição for excluído, o formulário de edição é limpo automaticamente.

Capturas de Tela

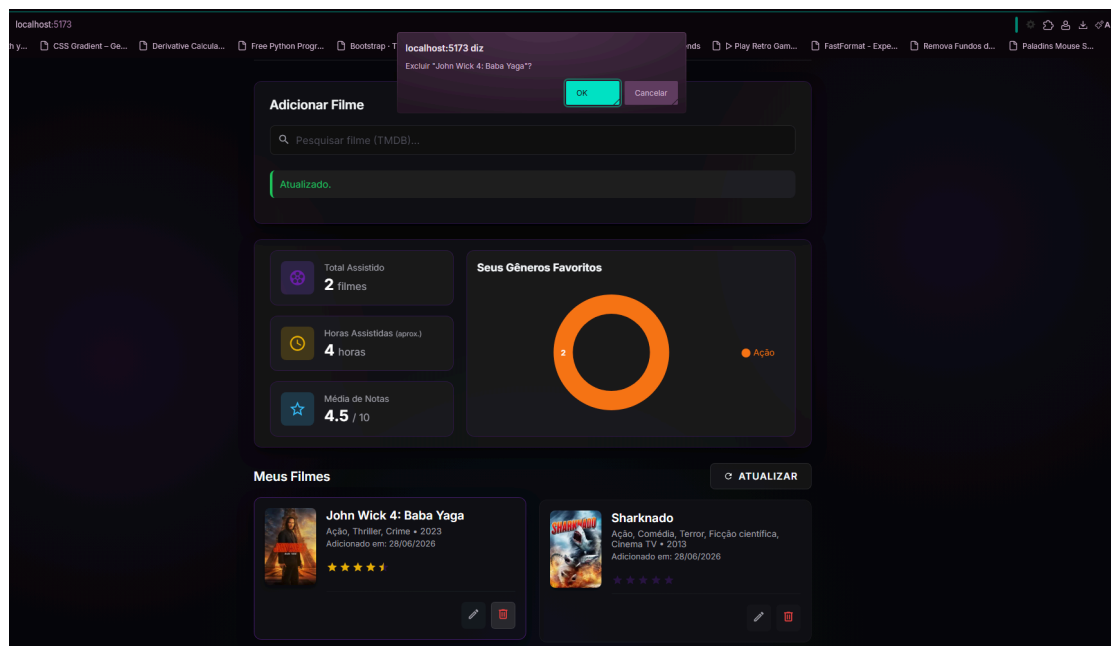


Figura 11 — Dialog de confirmação de exclusão do navegador

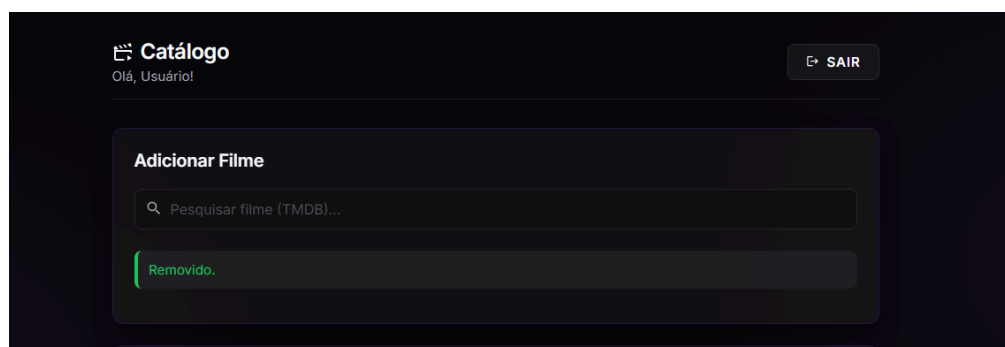


Figura 12 — Feedback "Removido." após exclusão bem-sucedida

5.4 Referência dos Endpoints da API

Todos os endpoints privados exigem o header HTTP: Authorization: Bearer (token_jwt)

5.4.1 Autenticação

POST /auth/register	Cria novo usuário. Campos: username, password
POST /auth/login	Autentica usuário e retorna access_token JWT

5.4.2 Filmes (privado)

GET /filmes	Retorna lista de filmes do usuário cadastrado
POST /filmes	Cadastra novo filme. Proprietário vinculado via JWT
PUT /filmes/{id}	Atualiza dados de um filme específico do usuário
DELETE /filmes/{id}	Remove filme do catálogo do usuário

5.4.3 Busca TMDB (privado)

GET /tmdb/search?query=...	Busca filmes na API do TMDB. Retorna até 10 resultados com título, gênero, ano, pôster e sinopse
-------------------------------	--

5.5 Estrutura do Banco de Dados

O banco de dados utilizado é o SQLite, com arquivo local filmes.db gerenciado pelo SQLAlchemy.

Tabela: usuarios	
id	INTEGER — Chave primária, autoincremento
username	TEXT — Nome de usuário único (UNIQUE)
hashed_password	TEXT — Senha criptografada com bcrypt
Tabela: filmes	
id	INTEGER — Chave primária, autoincremento
titulo	TEXT — Título do filme
genero	TEXT — Gêneros separados por vírgula
ano	INTEGER — Ano de lançamento
nota	REAL — Avaliação do usuário (0.0 a 10.0)
poster	TEXT — URL do pôster (TMDB)
usuario	TEXT — Nome do usuário proprietário
data_cadastro	TEXT — Data de adição ao catálogo

5.6 Resumo das Telas

ID	Tela	Acesso	Endpoint
TE	Login	Pública	POST /auth/login
LA-01			

ID	Tela		Acesso	Endpoint
LA-02	TE	Cadastro	Pública	POST /auth/register
LA-03	TE	Dashboard — Vazio	Privada	GET /filmes
LA-04	TE	Dashboard — Com Filmes	Privada	GET /filmes
LA-05	TE	Busca TMDb	Privada	GET /tmdb/search
LA-06	TE	Adicionar Filme	Privada	POST /filmes
LA-07	TE	Editar Filme	Privada	PUT /filmes/{id}
LA-08	TE	Excluir Filme	Privada	DELETE /filmes/{id}

6. Organização Ágil e Backlog

A equipe adotou princípios ágeis para organizar o desenvolvimento, dividindo o trabalho por camada da aplicação e acompanhando as entregas via GitHub Projects.

6.1 Divisão de Responsabilidades

Camada	/	Responsáveis	Status
Componente			
Model + Schema (Pydantic)	S.	Carlos A., Matheus	✓ Concluído

Repository (SQLAlchemy)	Bruno F., Eduardo R.	✓ Concluído
Service + Controller	Gabriel R., Gabriel V.	✓ Concluído
main.py + CORS + Database	Maurício G., Lucas R.	✓ Concluído
Frontend App.jsx + Estado Victor	João Vitor, João	✓ Concluído
Estilização CSS	Jamilly D., Guilherme R.	✓ Concluído
Docker + Docker Compose	Bruno L., Rafael M.	✓ Concluído
Documentação e README	Victor G.	✓ Concluído

6.2 Backlog de Funcionalidades

User Story	Prioridade	Status
Como usuário, quero cadastrar um filme (título, gênero, ano, nota).	Alta	✓ Concluído
Como usuário, quero listar todos os filmes cadastrados.	Alta	✓ Concluído
Como usuário, quero editar os dados de um filme existente.	Alta	✓ Concluído
Como usuário, quero excluir um filme com confirmação.	Alta	✓ Concluído

Como usuário, quero buscar filmes pelo título em tempo real.	Média	✓ Concluído
Como desenvolvedor, quero API documentada com Swagger.	Média	✓ Concluído
Como DevOps, quero executar com Docker Compose.	Média	✓ Concluído
Como usuário, quero feedback visual de sucesso e erro.	Média	✓ Concluído
Como usuário, quero autenticar com login e senha.	Baixa	⌚ Backlog
Como usuário, quero ver o pôster do filme via TMDb.	Baixa	⌚ Backlog

7. Versionamento e GitHub

O controle de versão do projeto foi realizado por meio do Git, com o código hospedado no GitHub, no repositório `APIRestfull---CRUD-de-Filmes` (<https://github.com/C4rl0s-ia/APIRestfull---CRUD-de-Filmes>). A plataforma foi utilizada não apenas como repositório de código, mas como ferramenta de colaboração entre os integrantes da equipe, permitindo o acompanhamento da evolução do projeto ao longo do desenvolvimento da API REST de gerenciamento de filmes (backend em FastAPI e frontend em React).

7.1 Padrão de Commits

Para manter um histórico organizado e de fácil compreensão, adotou-se a convenção `Conventional Commits`, na qual cada commit recebe um prefixo que indica a natureza da alteração realizada. Essa prática facilita a identificação do propósito de cada mudança sem a necessidade de abrir o código.

Os principais tipos utilizados foram:

Prefixo	Significado
feat	Adição de uma nova funcionalidade
fix	Correção de um bug ou erro
refactor	Reorganização do código sem alterar seu comportamento externo

Exemplos de commits realizados no projeto:

feat: criação da tela de login

fix: correção da validação de usuário

refactor: reorganização da camada de services

Ao todo, o repositório totaliza 30 commits, refletindo a evolução incremental do projeto desde a estruturação inicial do backend (camadas de controllers, services, repositories e models, seguindo princípios de Clean Architecture) até a implementação de funcionalidades como autenticação via JWT, integração com a API do TMDb e o painel analítico no frontend.

7.2 Pull Requests

Foram abertos 6 Pull Requests ao longo do desenvolvimento, utilizados como forma de integrar as alterações ao código principal de maneira organizada. Não foram utilizadas Issues para o rastreamento de tarefas, já que a divisão de trabalho entre os integrantes foi feita diretamente pela equipe, fora da plataforma.

7.3 Divisão de Trabalho na Equipe

Devido ao tamanho reduzido do escopo técnico em relação ao número de integrantes do grupo, a equipe optou por dividir as responsabilidades em duas frentes: uma parte dos integrantes ficou responsável pelo desenvolvimento da aplicação (código-fonte, versionado no GitHub), enquanto outra parte ficou responsável pela elaboração da documentação do projeto. Essa divisão explica por que o histórico de commits no repositório concentra-se em um número menor de integrantes, ainda que todo o grupo tenha contribuído de forma efetiva para a entrega final do trabalho.

8. Resultados

Ao final do desenvolvimento, a equipe entregou um sistema funcional que atende integralmente aos requisitos definidos no backlog. Seguem os principais resultados:

8.1 Funcionalidades Entregues

- API REST com cinco endpoints documentados e funcionais (POST, GET, GET/{id}, PUT, DELETE).
- Validação automática de dados com Pydantic, incluindo restrição de nota entre 0 e 10.
- Tratamento de erros HTTP padronizados: 404 (recurso inexistente) e 422 (dado inválido).
- Interface React com cadastro, listagem, edição, exclusão e busca em tempo real.
- Feedback visual imediato para todas as operações.
- Ambiente completo via Docker Compose com um único comando (docker compose up --build).
- Documentação automática da API via Swagger UI em /docs.

8.2 Indicadores de Qualidade

Métrica	Resultado
Endpoints implementados	5 de 5 (100%)
User stories concluídas	8 de 10 (80%)
Documentação Swagger	Gerada automaticamente pelo FastAPI

Conteínização	Backend + Frontend dockerizados
Tratamento de erros HTTP	404 e 422 implementados
Validação de entrada	Pydantic com Field constraints
Problema de infraestrutura	Identificado e documentado com solução

9. Conclusão

O projeto API RESTful de Cadastro de Filmes atingiu seus objetivos acadêmicos e técnicos. A equipe demonstrou capacidade de projetar e implementar uma aplicação web completa, integrando tecnologias modernas de backend, banco de dados e frontend, com foco em boas práticas de arquitetura e organização de código.

A separação em camadas (Controller, Service, Repository, Model, Schema) demonstra maturidade arquitetural e facilita a evolução do sistema. O SQLAlchemy garante segurança contra SQL Injection e centraliza o schema. O React com hooks modernos entrega uma interface reativa e responsiva. A conteínização resolve o problema de portabilidade de forma definitiva.

Do ponto de vista do processo, a equipe vivenciou um fluxo de desenvolvimento real: identificação de requisitos, implementação incremental, detecção e resolução de problemas técnicos reais (bug do Docker Compose com SQLite) e documentação formal. Essas experiências são fundamentais para a formação de engenheiros de software competentes.

Como próximos passos, destacam-se: implementação de autenticação JWT, migração para PostgreSQL, testes automáticos com pytest e integração com a API TMDb para enriquecer os dados com pôsteres e sinopses.

10. Referências

FASTAPI. FastAPI Documentation. Disponível em: <https://fastapi.tiangolo.com>. Acesso em: 21 jun. 2026.

SQLALCHEMY. SQLAlchemy Documentation — Version 2.0. Disponível em: <https://docs.sqlalchemy.org>. Acesso em: 21 jun. 2026.

PYDANTIC. Pydantic Documentation V2. Disponível em: <https://docs.pydantic.dev/latest>. Acesso em: 21 jun. 2026.

REACT. React — The library for web and native user interfaces. Disponível em: <https://react.dev>. Acesso em: 21 jun. 2026.

VITE. Vite — Next Generation Frontend Tooling. Disponível em: <https://vitejs.dev>. Acesso em: 21 jun. 2026.

DOCKER INC. Docker Documentation. Disponível em: <https://docs.docker.com>. Acesso em: 21 jun. 2026.

SQLITE. SQLite Documentation. Disponível em: <https://www.sqlite.org/docs.html>. Acesso em: 21 jun. 2026.

FIELDING, Roy Thomas. Architectural Styles and the Design of Network-based Software Architectures. Tese (Doutorado) — University of California, Irvine, 2000.

FOWLER, Martin. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2002.

ABNT NBR 6023:2018. Informação e documentação — Referências — Elaboração. Rio de Janeiro: ABNT, 2018.

ABNT NBR 14724:2011. Informação e documentação — Trabalhos acadêmicos — Apresentação. Rio de Janeiro: ABNT, 2011.